Next.js 16

Frontend · Meta-framework trên React · Next.js 16 (Node.js 20.9+, TypeScript 5.1+, React 19.2, Turbopack là bundler mặc định)

> **Trọng tâm của đề tài**
> Next.js là framework dựng trên React, bổ sung routing theo thư mục, render phía server, lấy dữ liệu và cache. Đề tài được đánh giá ở chỗ nhóm chỉ ra được **cái gì trong đồ án là của Next.js mà một ứng dụng React thuần (SPA) không làm giống như vậy**. Giao diện đẹp hay nhiều trang không tạo thêm điểm.

1. Bắt buộc — nhóm nào cũng phải tìm hiểu và làm được

A. Nền tảng Frontend (trình bày theo thuật ngữ của Next.js)

- **Component và props**: chia giao diện thành component tái sử dụng; phân biệt **Server Component** (mặc định trong App Router) và **Client Component** (khai báo `"use client"`).
- **State và data binding**: state cục bộ trong component (`useState`), state dùng chung (Context hoặc lưu trên URL bằng `searchParams`), và dữ liệu đến từ server. Với mỗi dữ liệu trong đồ án, nhóm nói được nó là loại nào và nằm ở đâu.
- **Vòng đời**: Server Component chỉ render trên server, không có `useEffect`; Client Component được hydrate rồi mới chạy effect. Giải thích hydration và nguyên nhân gây lỗi hydration mismatch.
- **Routing**: App Router, thư mục là route, `page` và `layout`, layout lồng nhau, điều hướng phía client bằng `next/link` mà không tải lại cả trang.
- **Lấy dữ liệu**: `fetch` hoặc truy vấn CSDL trực tiếp trong Server Component bất đồng bộ. Không biến mọi trang thành client component chỉ để gọi `fetch` trong `useEffect`.
- **Công cụ**: TypeScript; chạy bằng `next dev`, `next build`, `next start`; đọc được kết quả build (route nào tĩnh, route nào động). Turbopack là bundler mặc định ở bản 16.

B. Cơ chế riêng của Next.js 16

- **Ranh giới server/client**: component nào ở phía nào, vì sao; ranh giới đó ảnh hưởng tới kích thước JavaScript gửi xuống trình duyệt và dữ liệu nào bị lộ cho client.
- **API bất đồng bộ của bản 16**: `params`, `searchParams`, `cookies()`, `headers()` phải dùng với `await`. Code viết theo hướng dẫn cũ (truy cập đồng bộ) sẽ lỗi.
- **Mô hình cache mặc định**: ở bản 16, code động chạy lúc có request; cache chỉ có khi nhóm chủ động bật (Cache Components, `"use cache"`). Nhóm nói rõ đồ án của mình đang ở chế độ nào.
- **Luồng render của chính đồ án**: một request vào route cụ thể thì dữ liệu lấy ở đâu, phần nào render trên server, phần nào cần tương tác phía client.

2. Tự chọn theo ứng dụng — chỉ chọn cái ứng dụng của em thật sự cần

Không có số lượng cứng. Đã chọn mục nào thì phải giải thích được cơ chế, cấu hình và kết quả của mục đó; không chọn thì không cần nhét vào.

Cơ chế	Nên chọn khi	Cần chứng minh
Route động và lồng nhau; `loading.tsx`, `error.tsx`, `not-found.tsx`	Có trang danh sách và trang chi tiết	Trang chi tiết với id hợp lệ và không hợp lệ (hiện trang not-found); trạng thái loading khi dữ liệu chậm
Caching: `"use cache"`, `cacheLife`, `revalidateTag(tag, profile)`, `updateTag`	Dữ liệu ít đổi (danh mục, bài viết) hoặc cần cập nhật ngay sau khi sửa	So sánh trước và sau: số truy vấn CSDL hoặc thời gian phản hồi; tình huống dữ liệu đổi mà cache được làm mới đúng
Server Action hoặc Route Handler	Có form hoặc thao tác thay đổi dữ liệu	Chỉ rõ hàm chạy ở server; kiểm tra dữ liệu đầu vào ở server; làm mới dữ liệu sau khi ghi
Suspense và streaming	Có phần dữ liệu chậm	Vùng fallback hiện trước, dữ liệu hiện sau; ảnh hoặc video ghi lại
`next/image`, `next/font`, `generateMetadata`	Cần SEO hoặc nhiều ảnh	Cấu hình `images.remotePatterns`; kết quả Lighthouse trước và sau
`proxy.ts` (tên mới của `middleware.ts`)	Chuyển hướng hoặc kiểm tra đăng nhập đơn giản	Chỉ làm việc nhẹ ở đây; không đặt nghiệp vụ trong `proxy.ts`
Xác thực bằng cookie/session	Có đăng nhập	Route được bảo vệ ở phía server, không chỉ ẩn nút ở giao diện

3. Nâng cao — tự chọn để tạo chiều sâu

Không có số lượng cứng; nếu chọn, ưu tiên ít mục nhưng làm sâu, có minh chứng và có số liệu khi phù hợp. Một số mục (TLS, phân quyền, tương thích phiên bản…) chứng minh bằng test hoặc trace thay vì số liệu.

Hướng nâng cao	Làm gì và chứng minh bằng gì (số liệu, test hoặc trace)
Cache Components và Partial Prerendering	Bật `cacheComponents`; tách phần tĩnh và phần động của một trang bằng Suspense; đo TTFB hoặc LCP trước/sau và giải thích vì sao khác.
React Compiler (`reactCompiler: true`)	Cài `babel-plugin-react-compiler`; đo số lần re-render bằng React DevTools Profiler trước/sau trên một màn hình có danh sách lớn; ghi lại cả thời gian build tăng lên.
Đo và giảm JavaScript phía client	Dùng bundle analyzer; chuyển một client component sang server component và báo cáo kích thước bundle giảm bao nhiêu.
Kiểm thử tự động	Component test (Vitest + Testing Library) cho một component có state; E2E bằng Playwright cho luồng chính; chạy trong CI.
Triển khai	Docker với chế độ `standalone` hoặc nền tảng hosting; quản lý biến môi trường; README để nhóm khác chạy lại được.

4. Minh chứng cần có trong báo cáo và repo

- Chỉ ra ít nhất một route thật trong đồ án và giải thích ranh giới server/client của route đó (kèm file và dòng code).
- Ảnh hoặc log chứng minh dữ liệu được lấy đúng phía đã thiết kế (ví dụ log server khi tải trang; tab Network không còn request lấy dữ liệu đó từ trình duyệt).
- Nếu dùng cache, `revalidateTag`/`updateTag`, Server Action hoặc Route Handler: có tình huống chạy trước và sau thay đổi.
- Bảng phủ kỹ thuật: mỗi mục ở phần 1 và 2 đã làm trỏ tới file hoặc commit cụ thể.

5. Chưa được xem là đạt nếu chỉ…

- Tạo nhiều trang React đặt trong thư mục `app` nhưng toàn bộ đều là Client Component.
- Dùng Next.js chỉ như bộ router, còn dữ liệu vẫn lấy kiểu SPA mà không giải thích được vì sao.
- Kể tên SSR, SSG, ISR nhưng không chỉ ra được cơ chế render hay cache nào đồ án đang thật sự dùng.
- Chỉ chạy `create-next-app` rồi sửa nội dung trang mặc định.

6. Câu hỏi tự kiểm

- Cái gì trong đồ án này là Next.js mà một React SPA bình thường không làm giống như vậy?
- Nếu bỏ `"use client"` ở component này thì chuyện gì xảy ra? Vì sao?
- Dữ liệu này nên cache hay luôn lấy mới? Quyết định đó ảnh hưởng gì tới người dùng?
- Trang này render ở đâu và vào lúc nào? Nhìn vào đâu để chứng minh điều đó?

7. So sánh và phản biện (để viết phần Tổng quan)

Công nghệ thay thế và lý do chọn Next.js

- **React + Vite (SPA)**: đơn giản hơn, hợp ứng dụng sau đăng nhập không cần SEO; chọn Next.js khi cần render phía server, SEO, hoặc muốn giảm JavaScript gửi xuống.
- **Nuxt 4, SvelteKit, Angular (SSR)**: cùng ý tưởng ở hệ khác; chọn Next.js phải có lập luận riêng của nhóm (ví dụ hệ sinh thái React, thư viện sẵn có), không chỉ nói "phổ biến hơn".
- **Astro**: tốt cho trang nội dung tĩnh, ít tương tác; Next.js hợp hơn khi ứng dụng có nhiều trạng thái và tương tác.

Những trường hợp KHÔNG nên dùng Next.js (nêu ít nhất 2, dựa trên trải nghiệm của nhóm)

- Ứng dụng nội bộ sau đăng nhập, không cần SEO, ít trang: một SPA React + Vite đủ dùng và ít khái niệm hơn.
- Cần giữ kết nối thời gian thực lâu dài (WebSocket nhiều người dùng): nên có dịch vụ backend riêng; Next.js không phải nơi lý tưởng cho việc này.
- Nhóm chỉ cần một vài trang giới thiệu tĩnh: Astro hoặc HTML thuần nhẹ hơn nhiều.

8. Ví dụ bài thực hành tại chỗ

Chỉ là ví dụ. Đề chính thức được ra từ chính đồ án nhóm đã nộp.

- Thêm một dynamic route vào module có sẵn và xử lý trường hợp không tìm thấy.
- Chuyển một thao tác cập nhật từ gọi `fetch` ở client sang Server Action hoặc Route Handler.
- Giải thích rồi chỉnh lại một ranh giới server/client chưa hợp lý trong chính đồ án.